Optimal Control HW5

Brandon Lim

March 2, 2026

ME EN 7210 Optimal Controls **Spring 2026**
Assignment #5 **Due Tuesday, Mar. 3 at the beginning of class**

Problem 1 (Exact system inversion problem)

(a) Consider the following piezo-base positioning system:

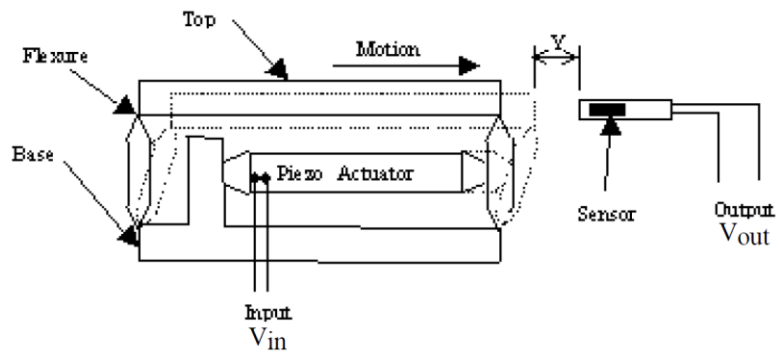


Figure 1: Schematic of piezo-stage

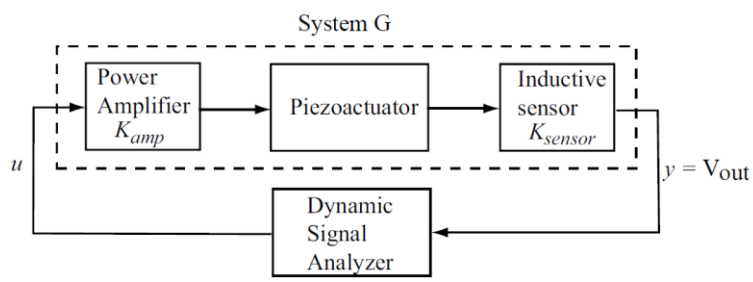


Figure 2: Modeling the vibrational dynamics using a dynamic signal analyzer

The piezo-stage is a positioning system that can be used in applications that require small (tens of micrometers) displacements. The piezo-stage consists of a piezo-actuator, flexural mechanism and a sensor as shown in Figure 1. An input voltage is applied to a piezo-actuator, which expands and pushes against the flexural mechanism that amplifies the piezo-movement. The displacement of the piezo-stage is measured using and inductive sensor. The inductive sensor has a gain of 0.2 volts/µm and measures a output voltage (VOUT). Figure 2 shows the actual system itself connected to an instrument called a Dynamic Signal Analyzer to measure the system's frequency response (Bode plot), which is then used to determine a transfer function. Afterwards, the transfer function is converted to a state-space model (in control canonical form) given by:

$$\dot{x} = \begin{bmatrix} -1.169 & -50.3 & -45.94 & -685.3 & -391.7 & -1952 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} x + \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} u$$

$$y = \begin{bmatrix} 0 & 11.88 & 4.977 & 539.6 & 129.9 & 5625 \end{bmatrix} x$$

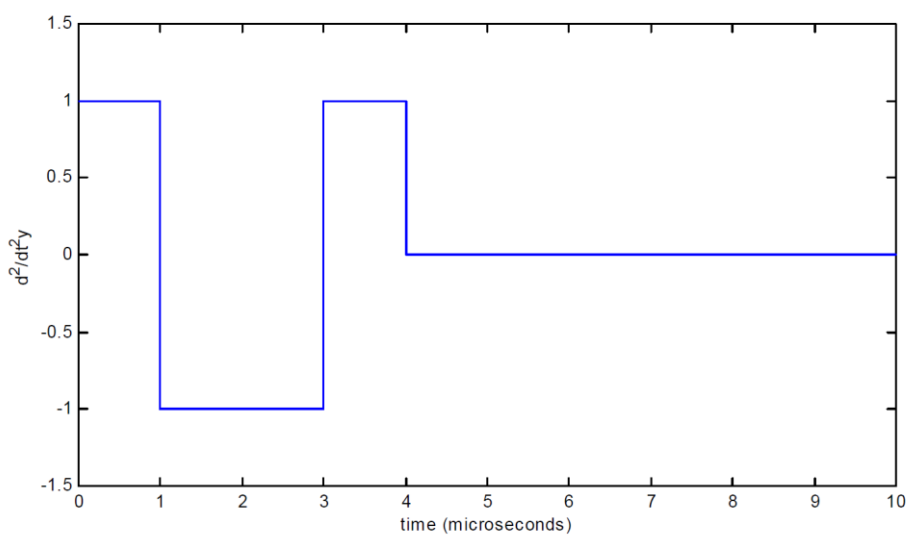
$$\dot{x} = Ax + Bu$$

$$y = Cx$$

Determine the relative degree of the system. Note that relative degree means the difference in the order between the numerator and denominator polynomial of the system transfer function.

(b) Determine the state-space model of the inverse system given some desired output trajectory $y_d(t)$ that you want to track using the system inversion technique. What are the eigenvalues of the inverse system and how do they compare to the eigenvalues of the original system? You may want to compare the poles of the original system transfer function to the inverse system transfer function to gain some additional insight.

(c) Simulate the response of the inverse system model using MATLAB (lsim command) to find the inverse feedforward input. The desired acceleration profile is shown below.



In your simulation, show plots of the $y_d(t)$ and its first time derivative, and also show the response of the inverse system, that is the behavior of the inverse states (reference states per the notes in class).

(d) From the results above, determine the feedforward input and show plots of the input vs. time. Apply this feedforward input to the system model given above to verify that output

tracking is achieved (use lsim command again) --- do not use state feedback! Show plots of the desired output $y_d(t)$ compared to the output of the system when you applied the inverse input you determined. Does the system track the desired output using the inverse feedforward input?

Problem 2 (Optimal system inversion proof)

In class, Dr. Leang lectured on the frequency-based optimal inversion process and gave the essential equations for the optimal input and optimal output trajectories that minimize the cost function:

$$J(U) = \int_{-\infty}^{\infty} \left[U^*(j\omega)R(j\omega)U(j\omega) + e^*(j\omega)Q(j\omega)e(j\omega)\right] d\omega,$$

where `*' denotes the complex conjugate transpose, $U(j\omega)$ is the feedforward input to the system, $e(j\omega) = Y(j\omega) - Yd(j\omega)$ is the tracking error, $Y(j\omega)$ is the output, $Yd(j\omega)$ is the desired output, and $R(j\omega)$ and $Q(j\omega)$ are non-negative, frequency-dependent real-value weights on the input energy and the tracking error, respectively.

Show that for an open-loop system with model $G(j\omega)$ the optimal input and output are given by:

$$U_{opt}(j\omega) = \left[\frac{G^*(j\omega)Q(j\omega)}{R(j\omega) + G^*(j\omega)Q(j\omega)G(j\omega)}\right] Y_d(j\omega).$$

$$\begin{aligned} Y_{opt}(j\omega) &= G(j\omega)u_{opt}(j\omega), \\ &= G(j\omega)\left[\frac{G^*(j\omega)Q(j\omega)}{R(j\omega) + G^*(j\omega)Q(j\omega)G(j\omega)}\right] Y_d(j\omega), \\ &\triangleq G_f(j\omega)Y_d(j\omega). \end{aligned}$$

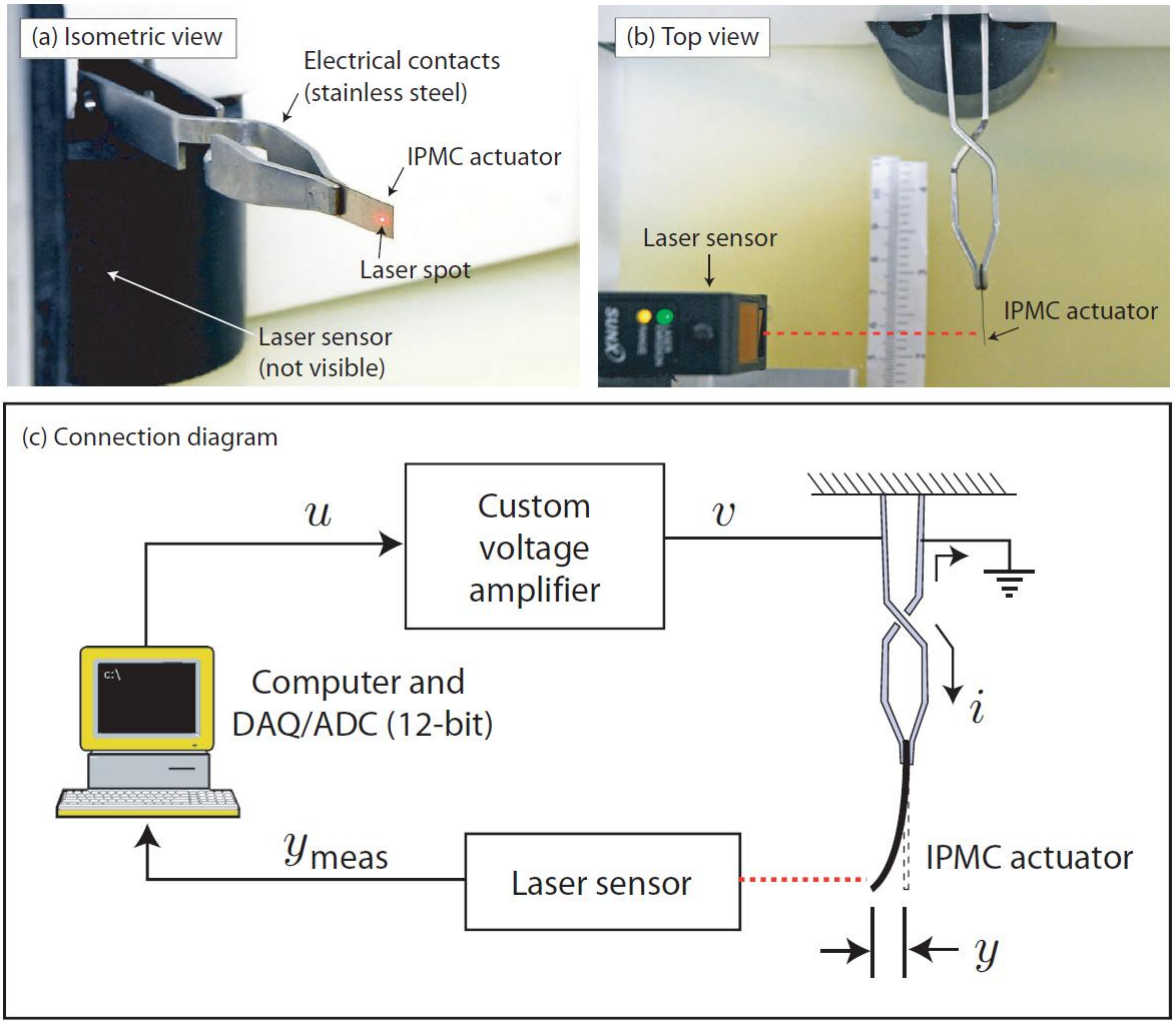
Hint: compute and solve the following:

$$\frac{dJ(U)}{dU} = 0,$$
$$\frac{d^2J(U)}{dU^2} > 0.$$

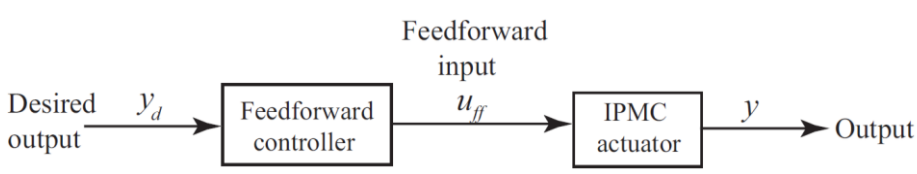
Problem 3 (Optimal system inversion implementation on IPMC actuator system)

In this problem, you will implement the frequency-based optimal inversion approach to feedforward control an example smart-material actuator, specifically an ionic-polymer metal

composite (IPMC) actuator. An IPMC actuator is a type of smart material where an input voltage causes it to displace, creating motion:



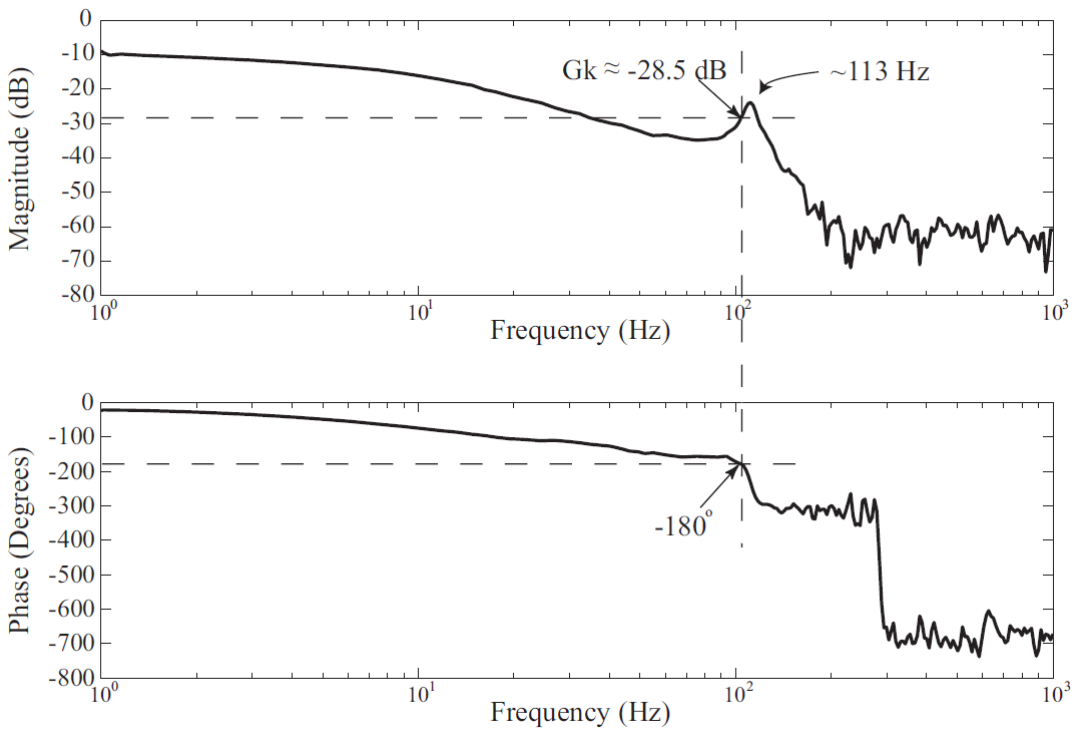
The control diagram is shown below where your job is to final the optimal feedforward input u_{opt}, which is essentially a modified version of the exact feedforward input u_{ff}.



The equations you'll implement is given above in Problem 2 and the optimal input and output will minimize the cost function:

$$J(U) = \int_{-\infty}^{\infty} \left[U^*(j\omega)R(j\omega)U(j\omega) + e^*(j\omega)Q(j\omega)e(j\omega)\right] d\omega,$$

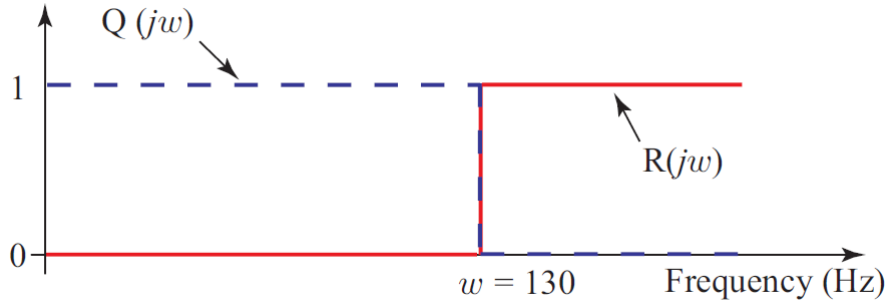
The model of the IPMC actuator was obtained experimentally using a dynamic signal analyzer that measures the system's Bode plot (frequency response):



The frequency response plots for $G(j\omega)$ of the IPMC actuator is shown above, where the Gk is the gain margin.

For the optimal inversion process, you will use $G(j\omega)$ above, along with using a triangular trajectory with different frequencies: 1Hz, 5Hz, 10Hz, and 18Hz. You will determine optimal inputs and outputs for these different trajectories and compare the results.

Pick the weights as follows: $Q = 1$ and $R = 0$ for $\omega \in [0; 130]$ Hz and $Q = 0$ and $R = 1$ for $\omega > 130$ Hz as shown below. The weights were chosen such that the computed feedforward input magnitude was less than 5 V.

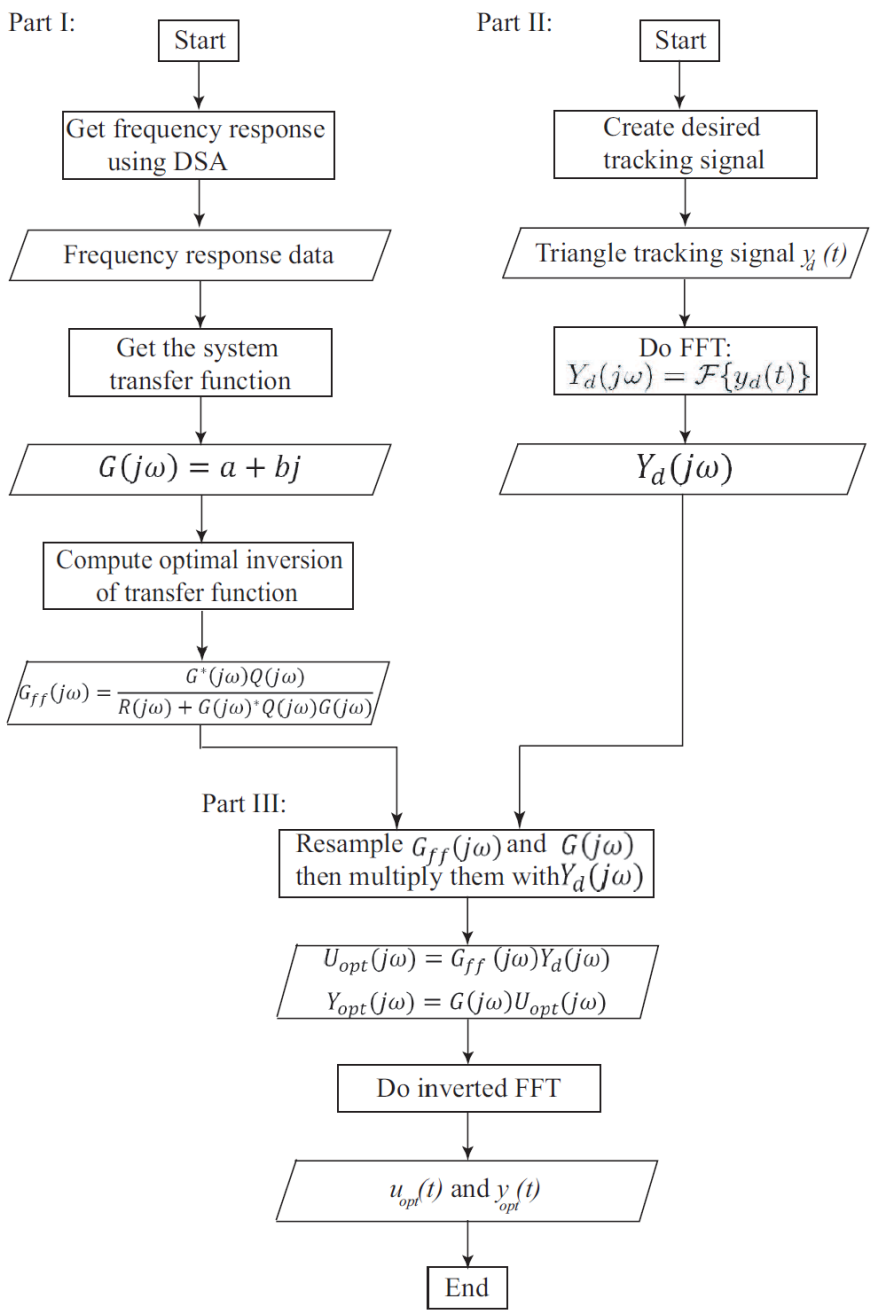


Instructions: Download and extract the Matlab M-file with supporting data files, filename "OptimalInvStart.zip". The extracted files contains the following files:

- Main.m – main m-file that loads the desired trajectory, which is a triangle signal of 1 Hz. Other frequencies are included, such as "yd05.in" for 5Hz, "yd10.in" for 10 Hz, "yd18.in" for 18 Hz. The main.m file shows an example for 1Hz, so you will change the load command to load trajectories for other frequencies (see details below).
- The files "cm.txt" and 'cp.txt" are the magnitude and phase data for the system model, and "cm.x: is the file for the frequency range. The main.m file when you run it will generate the plots for the desired trajectory and the frequency-response model G(jω). You will use these plots in creating the optimal inversion input and output.
- The process to implement the frequency-based optimal inversion process is outlined below, so follow the steps. In Part I, the steps "Get frequency response data using DSA", "Frequency response data", "Get the system transfer function", and "G(jω) = a+bj" is already done in the main.m file. Also in Part II, "Create desired tracking signal" and "Triangle tracking signal $y_d(t)$" is also already done in the main.m file, so please look it over so you understand what it's doing. Your job is to perform the rest of the steps in Part I, II and III. Follow the flow diagram as you will need to implement the FFT on the desired triangular trajectory $y_d(t)$ to get $Y_d(j\omega)$, then compute $U_{opt}(j\omega)$ and $Y_{opt}(j\omega)$, followed by an inverse FFT to find $u_{opt}(t)$ and $y_{opt}(t)$.

Show:

- Plots of the optimal input and output for the 1 Hz triangular signal. Because the trajectory has many cycles, only plot 2-3 cycles when the behavior reaches steady state. Compare the output with the actual desired trajectory, so plot $y_d(t)$ and $y_{opt}(t)$ together, as well as the optimal input $u_{opt}(t)$. Note that you need to compute $u_{opt}(t)$ and apply it to your system model to verify that it's working correctly.
- Do the same for 5 Hz, 10 Hz, and 18 Hz. Compare the results and describe what is happening as the frequency increases. Relate back to the choice of the Q and R weightings as a function of frequency.
- Submit your Matlab code.



The process for the feedforward input computation. Part I was used to treat the model $G(j\omega)$ of the IPMC actuator system and to find the optimal inversion from the system model $Gff(j\omega)$. This part was based on the $Q(j\omega)$ and $R(j\omega)$ weights in frequency domain. Part II was used to treat the given desired tracking trajectory $yd(t)$, *i.e.*, to convert to the frequency domain $Yd(j\omega)$. Finally, in Part III the frequency response function $G(j\omega)$ and $Gff(j\omega)$ from Part I along with the $Yd(j\omega)$ from Part II were used to find the time domain signal $u_{opt}(t)$ and $y_{opt}(t)$.

Problem 1

All code given in the appendix.

a)
From Matlab:

$$G(S) = \frac{11880s^4 + 4977s^3 + 539600s^2 + 129900s + 5625000}{1000s^6 + 1169s^5 + 50300s^4 + 49940s^3 + 685300s^2 + 391700s + 1952000}$$

Relative degree = r = 2

b)

$$u_{ff} = (CA^{(r-1)}B)^{-1}(y_d^r - CA^r x_{ref})$$

$$u_{ff} = (CAB)^{-1}(\ddot{y}_d - CAAx_{ref})$$

$$\dot{x}_{ref} = Ax_{ref} + Bu_{ff}$$

$$\dot{x}_{ref} = Ax_{ref} + B(CAB)^{-1}(\ddot{y}_d - CAAx_{ref})$$

$$\dot{x}_{ref} = \underbrace{(A - B(CAB)^{-1}CAA)}_{\hat{A}} x_{ref} + \underbrace{B(CAB)^{-1}}_{\hat{B}} \ddot{y}_d$$

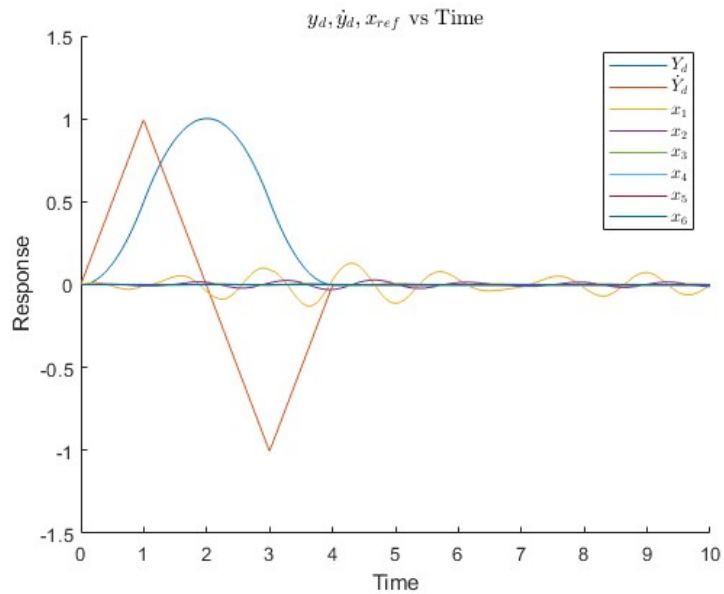
Using Matlab:

$$\dot{x}_{ref} = \begin{bmatrix} -0.4189 & -45.42 & -10.93 & -473.5 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} x_{ref} + \begin{bmatrix} 0.08418 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \ddot{y}_d$$

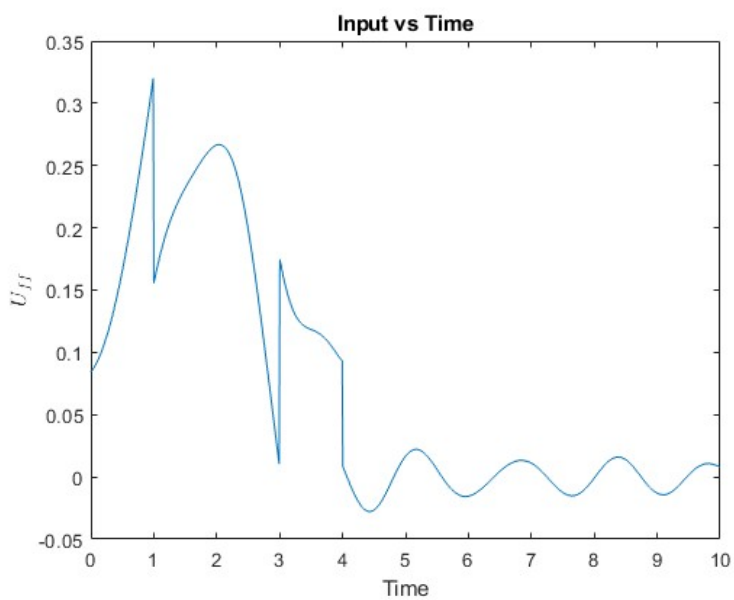
$$\lambda_{old} = \begin{bmatrix} -0.0666 + 5.3612i \\ -0.0666 - 5.3612i \\ -0.1709 + 4.1485i \\ -0.1709 - 4.1485i \\ -0.3470 + 1.9541i \\ -0.3470 - 1.9541i \end{bmatrix}, \lambda_{new} = \begin{bmatrix} 0.0000 + 0.0000i \\ 0.0000 + 0.0000i \\ -0.1601 + 4.0277i \\ -0.1601 - 4.0277i \\ -0.0494 + 5.3981i \\ -0.0494 - 5.3981i \end{bmatrix}$$

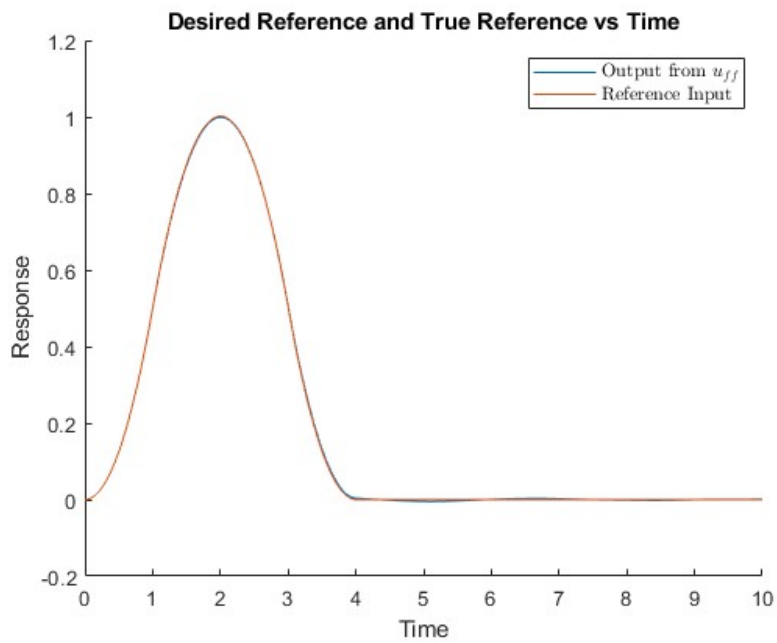
The eigenvalues of the old system all have negative real parts which means that the system is stable. The eigenvalues of the new system now have 2 eigenvalues with real parts that are equal to zero meaning the system is now marginally stable.

c)


$y_d, \dot{y}_d, x_{ref}$ vs Time
Response
Time
Y_d
$\dot{Y}_d$
x_1
x_2
x_3
x_4
x_5
x_6

d)


Input vs Time
U_{ff}
Time



The system does track the desired output using inverse feedforward.

Problem 2

$J(U) = \int_{-\infty}^{\infty} U^*(jw)R(jw)U(jw) + e^*(jw)Qe(jw)dw$

$e(jw) = Y(jw) - Y_d(jw)$

$J(U) = \int_{-\infty}^{\infty} U^*(jw)R(jw)U(jw) + (Y(jw) - Y_d(jw))^*Q(jw)(Y(jw) - Y_d(jw))dw$

let $Y(jw) = G(jw)U(jw)$

$J(U) = \int_{-\infty}^{\infty} U^*(jw)R(jw)U(jw) + (G(jw)U(jw) - Y_d(jw))^*Q(jw)(G(jw)U(jw) - Y_d(jw))dw$

$J(U) = \int_{-\infty}^{\infty} U^*(jw)R(jw)U(jw) + (G^*(jw)U^*(jw) - Y_d^*(jw))Q(jw)(G(jw)U(jw) - Y_d(jw))dw$

$\frac{dJ(U)}{dU} = \int_{-\infty}^{\infty} U^*(jw)R(jw) + G^*(jw)U^*(jw)Q(jw)G(jw) - Y_d^*(jw)Q(jw)G(jw)dw = 0$

Must hold for all w from $-\infty : +\infty$ therefore:

$U^*(jw)R(jw) + G^*(jw)U^*(jw)Q(jw)G(jw) - Y_d^*(jw)Q(jw)G(jw) = 0$

$U^*(jw)(R(jw) + G^*(jw)Q(jw)G(jw)) = Y_d^*(jw)Q(jw)G(jw)$

$[U^*(jw)(R(jw) + G^*(jw)Q(jw)G(jw))]^* = [Y_d^*(jw)Q(jw)G(jw)]^*$

Identity: $(ABC)^* = B^*A^*C^*$

$(R(jw) + G^*(jw)Q(jw)G(jw))^*U(jw) = G(jw)^*Q^*(jw)Y_d(jw)$

$(R^*(jw) + G(jw)Q^*(jw)G^*(jw))U(jw) = G(jw)^*Q^*(jw)Y_d(jw)$

$Q^* = Q, R^* = R$

$(R(jw) + G(jw)Q(jw)G^*(jw))U(jw) = G(jw)^*Q(jw)Y_d(jw)$

$$\boxed{U_{opt}(jw) = \frac{G(jw)^*Q(jw)}{(R(jw) + G^*(jw)Q(jw)G(jw))}Y_d(jw)}$$

Recall: $Y(jw) = G(jw)U(jw)$

$Y_{opt}(jw) = G(jw)U_{opt}(jw)$

$$\boxed{Y_{opt}(jw) = G(jw)\frac{G(jw)^*Q(jw)}{(R(jw) + G^*(jw)Q(jw)G(jw))}Y_d(jw)}$$

$Y_{opt}(jw) = G_f(jw)Y_d(jw)$

Checking Extrema:

$\frac{d^2J(U)}{dU^2} = 0 \rightarrow$ Weak minimum further investigation needed

Problem 3

All code given in the appendix.

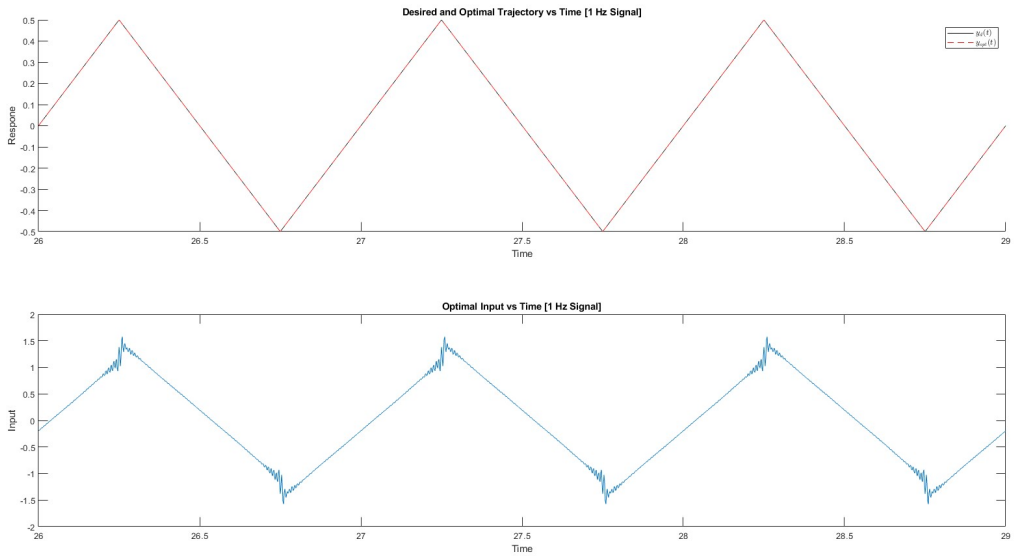


Figure 1: 1 Hz Desired Trajectory

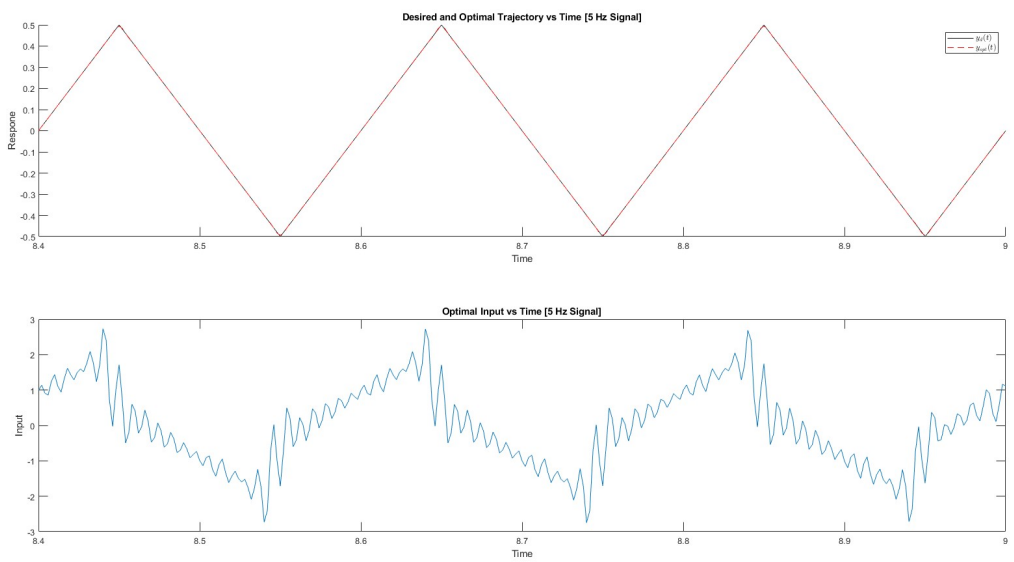


Figure 2: 5 Hz Desired Trajectory

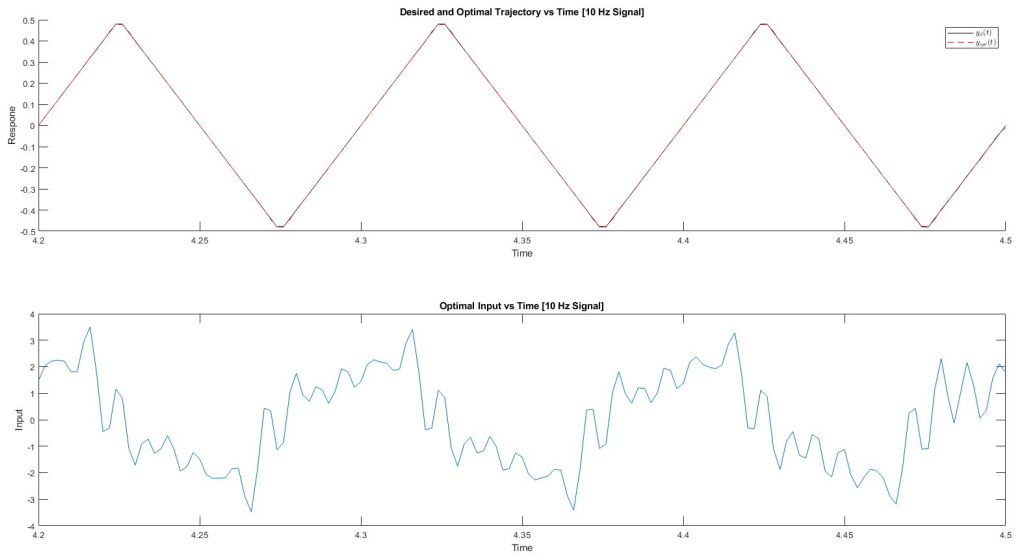


Figure 3: 10 Hz Desired Trajectory

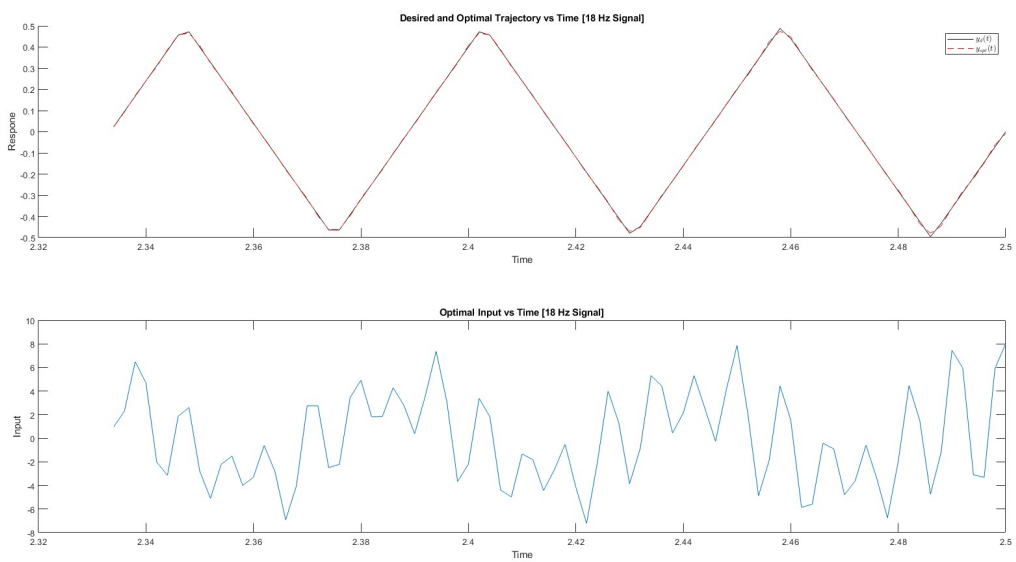


Figure 4: 18 Hz Desired Trajectory

From comparing the different trajectory frequencies, it is apparent that the input continues to grow in amplitude and oscillations to keep up with the frequency changes of the desired trajectory. Relating this back to the choice of weightings for Q and R, when the frequency gets high enough, we want to keep the input low and beneath a constraint value. So, we switch the weighting on the input to high and the weighting on the tracking error to low when we are at high enough of a frequency to where we need to meet our system input constraint. As a result, the tracking trajectory is less accurate as the frequency increases and the sharp peaks of the desired trajectory are attenuated on the actual trajectory. The reflection of weightings on the input can be seen for these higher frequency changes in the 18hz case clearly.

At the peaks of the triangles in the tracking trajectory where the frequency is high, the input can be seen to drop down due to the higher weighting.

```
clear,clc,close all
syms s

%Original System
A = [-1.169 -50.3 -45.94 -685.3 -391.7 -1952;
    1 0 0 0 0 0;
    0 1 0 0 0 0;
    0 0 1 0 0 0;
    0 0 0 1 0 0;
    0 0 0 0 1 0];
B = [1;0;0;0;0;0];
C = [0 11.88 4.977 539.6 129.9 5625];
D = 0;
SI = [s 0 0 0 0 0;
    0 s 0 0 0 0;
    0 0 s 0 0 0;
    0 0 0 s 0 0;
    0 0 0 0 s 0;
    0 0 0 0 0 s];

%Transfer Function Calculation
Transfer_Function_Orig = simplify(C*(inv(SI-A))*B);

%Inverted A and B matricies for ff control
A_new = A-(B*inv(C*A*B)*C*A*A);
B_new = B*(inv(C*A*B));

%State Space Forms
sys_new = ss(A_new,B_new,C,D);
sys_org = ss(A,B,C,D);

%Eigen Values
eig(A)
eig(A_new);

%New Transfer Function
Transfer_Function_New = simplify(C*(inv(SI-A_new))*B_new);


%Acceleration Reference Input
t = linspace(0,10,1000);
yddot(t>=0 & t<1) = 1;
yddot(t>=1 & t<3) = -1;
yddot(t>=3 & t<4) = 1;
yddot(t>=4 ) = 0;

ydot = cumtrapz(t,yddot);
plot(t,yddot)


[yd, t, xref] = lsim(sys_new,yddot,t);
```

```
figure
hold on
plot(t, yd)
plot(t,ydot)
plot(t,xref)
legend("$Y_d$", "$\dot{Y}_d$", "$x_1$", "$x_2$", "$x_3$", "$x_4$", "$x_5$", "$x_6$","↙
Interpreter", "Latex")
xlabel("Time"); ylabel("Response"); title("$y_d, \dot{y}_d, x_{ref}$ vs Time",↙
"Interpreter","latex")

uff = inv(C*A*B) .* (yddot - C*A*A*xref');
figure
plot(t,uff)
xlabel("Time"); ylabel("$U_{ff}$", "Interpreter","latex"); title("Input vs Time")

[yopt,t,x] = lsim(sys_org,uff,t);
figure
hold on
plot(t,yopt)
plot(t,yd)
xlabel("Time"); ylabel("Response"); title("Desired Reference and True Reference vs Time")
legend("Output from $u_{ff}$", "Reference Input", "Interpreter","latex")
```

```
clear, clc, close all
% load desired input trajectory signal---------
load yd01.in % trajectory for 1 Hz
yd = yd01(1:29999);
SignalFreq = 1; % Hz
SamplingFreq = 500; % Hz
dt = 1/SamplingFreq;
t = dt*[0:length(yd)-1];% time(s)--signal length
figure(1);clf;
subplot(211); plot(t,yd); xlabel('time (s)'); ylabel('y_d')

%=====================================================
% system modeling for (Transfer Function) G
%system modeling for (Transfer Function) G
%=====================================================
w = load('cm.x'); % load frequency
mag = load('cm.txt'); % load mag in dB
ph = load('cp.txt'); % ph in deg.
wn = find(w>130); % wn = find(w>133);
w = w(1:wn(1));
mag = mag(1:wn(1));
ph = ph(1:wn(1));

figure(2);clf;
subplot(211); semilogx(w,mag,'k'); xlabel('Freq. (Hz)'); ylabel('Mag. (dB)'); hold on
subplot(212); semilogx(w,ph,'k'); xlabel('Freq. (Hz)'); ylabel('Ph. (deg)');hold on

%complex freq response G (Transfer Function)-----
i = sqrt(-1);
for k =1:length(w)
    a = 10^(mag(k)/20)*cos(ph(k)*pi/180);
    b = 10^(mag(k)/20)*sin(ph(k)*pi/180);
    G(k,1) = a+b*i; % the Transfer Function of system
end

% make G even and get G_jq (conjugate of G)------
w = [0;w];
G = [G(1); G]; % Even G
G_jq = conj(G); % The conjugate of the even TF (G)
ome_g = w*2*pi; % get the omega of G_ff and G

% Create your optimal inversion code below -- please follow the process flow described in
homework.
%Step 1
%Compute Optimal Inversion of Transfer Function
Q(0<=w & w<130) = 1;
Q(w>130) = 0;
R(0<w & w<130) = 0;
R(w>130) = 1;
Q = Q';
R = R';
```

```
%Calculate Optimal Inversion Transfer Function
for i = 1:length(w)
    Gff(i,1) = (G_jq(i)*Q(i))/(R(i) + G_jq(i)*Q(i)*G(i));
end

% Step 2
% Cleaning up Yd for fft
I1 = find(t == 15);
I2 = find(t == 45);
yd = yd(I1:I2);

[Mag, Phase, freq, Yd_FFT] = fft_sample(yd, 500);
figure
plot(freq,Mag)

G_interp = interp1(ome_g, G, freq*2*pi, "linear",0);
Gff_interp = interp1(ome_g, Gff, freq*2*pi, "linear",0);

G_Full = [G_interp,flip(conj(G_interp(1:end-1)))]';
Gff_Full = [Gff_interp,flip(conj(Gff_interp(1:end-1)))]';

Uopt_jw = Gff_Full .* Yd_FFT;
Yopt_jw = G_Full .* Uopt_jw;

uopt = real(ifft(Uopt_jw));
yd_opt = real(ifft(Yopt_jw));

cycIdx1 = find(t==26);
cycIdx2 = find(t==29);

figure
subplot(2,1,1)
hold on
plot(t(cycIdx1:cycIdx2), yd(cycIdx1:cycIdx2), "k")
plot(t(cycIdx1:cycIdx2), yd_opt(cycIdx1:cycIdx2), "--r")
legend("$y_d(t)$", "$y_{opt}(t)$", "Interpreter", "Latex")
xlabel("Time"); ylabel("Respone"); title("Desired and Optimal Trajectory vs Time [1 Hz
Signal]")
subplot(2,1,2)
plot(t(cycIdx1:cycIdx2),uopt(cycIdx1:cycIdx2))
xlabel("Time"); ylabel("Input"); title("Optimal Input vs Time [1 Hz Signal]")

%% 5Hz
clear, clc, close all
% load desired input trajectory signal---------
load yd01.in % trajectory for 1 Hz
yd = yd01(1:29999);
SignalFreq = 5; % Hz
SamplingFreq = 500; % Hz
dt = 1/SamplingFreq;
```

```
t = dt*[0:length(yd)-1];% time(s)--signal length
yd = interp1(t, yd, t.*SignalFreq, 'linear', 0);
figure(1);clf;
subplot(211); plot(t,yd); xlabel('time (s)'); ylabel('y_d')

tstart = 15/SignalFreq;
tend = 45/SignalFreq;

I1 = find(t==tstart);
I2 = find(t==tend);
t = t(I1:I2);
yd = yd(I1:I2);


%=====================================================
% system modeling for (Transfer Function) G
%system modeling for (Transfer Function) G
%=====================================================
w = load('cm.x'); % load frequency
mag = load('cm.txt'); % load mag in dB
ph = load('cp.txt'); % ph in deg.
wn = find(w>130); % wn = find(w>133);
w = w(1:wn(1));
mag = mag(1:wn(1));
ph = ph(1:wn(1));

figure(2);clf;
subplot(211); semilogx(w,mag,'k'); xlabel('Freq. (Hz)'); ylabel('Mag. (dB)'); hold on
subplot(212); semilogx(w,ph,'k'); xlabel('Freq. (Hz)'); ylabel('Ph. (deg)');hold on

%complex freq response G (Transfer Function)-----
i = sqrt(-1);
for k =1:length(w)
    a = 10^(mag(k)/20)*cos(ph(k)*pi/180);
    b = 10^(mag(k)/20)*sin(ph(k)*pi/180);
    G(k,1) = a+b*i; % the Transfer Function of system
end

% make G even and get G_jq (conjugate of G)------
w = [0;w];
G = [G(1); G]; % Even G
G_jq = conj(G); % The conjugate of the even TF (G)
ome_g = w*2*pi; % get the omega of G_ff and G

% Create your optimal inversion code below -- please follow the process flow described in
homework.
%Step 1
%Compute Optimal Inversion of Transfer Function
Q(0<=w & w<130) = 1;
Q(w>130) = 0;
R(0<w & w<130) = 0;
```

```
R(w>130) = 1;
Q = Q';
R = R';

%Calculate Optimal Inversion Transfer Function
for i = 1:length(w)
    Gff(i,1) = (G_jq(i)*Q(i))/(R(i) + G_jq(i)*Q(i)*G(i));
end

% Step 2
% Cleaning up Yd for fft

[Mag, Phase, freq, Yd_FFT] = fft_sample(yd, 500);
figure
plot(freq,Mag)

G_interp = interp1(ome_g, G, freq*2*pi, "linear",0);
Gff_interp = interp1(ome_g, Gff, freq*2*pi, "linear",0);

G_Full = [G_interp,flip(conj(G_interp(1:end-1)))];
Gff_Full = [Gff_interp,flip(conj(Gff_interp(1:end-1)))];

Uopt_jw = Gff_Full .* Yd_FFT;
Yopt_jw = G_Full .* Uopt_jw;

uopt = real(ifft(Uopt_jw));
yd_opt = real(ifft(Yopt_jw));

numSec = 3/SignalFreq;

cycIdx1 = find(t==(tend-numSec));
cycIdx2 = find(t==(tend));

figure
subplot(2,1,1)
hold on
plot(t(cycIdx1:cycIdx2), yd(cycIdx1:cycIdx2), "k")
plot(t(cycIdx1:cycIdx2), yd_opt(cycIdx1:cycIdx2), "--r")
legend("$y_d(t)$", "$y_{opt}(t)$", "Interpreter", "Latex")
xlabel("Time"); ylabel("Respone"); title("Desired and Optimal Trajectory vs Time [5 Hz
Signal]")
subplot(2,1,2)
plot(t(cycIdx1:cycIdx2),uopt(cycIdx1:cycIdx2))
xlabel("Time"); ylabel("Input"); title("Optimal Input vs Time [5 Hz Signal]")

%% 10Hz
clear, clc, close all
% load desired input trajectory signal---------
load yd01.in % trajectory for 1 Hz
yd = yd01(1:29999);
SignalFreq = 10; % Hz
```

```
SamplingFreq = 500; % Hz
dt = 1/SamplingFreq;
t = dt*[0:length(yd)-1];% time(s)--signal length
yd = interp1(t, yd, t.*SignalFreq, 'linear', 0);
figure(1);clf;
subplot(211); plot(t,yd); xlabel('time (s)'); ylabel('y_d')

tstart = 15/SignalFreq;
tend = 45/SignalFreq;

I1 = find(t==tstart);
I2 = find(t==tend);
t = t(I1:I2);
yd = yd(I1:I2);


%=====================================================
% system modeling for (Transfer Function) G
%system modeling for (Transfer Function) G
%=====================================================
w = load('cm.x'); % load frequency
mag = load('cm.txt'); % load mag in dB
ph = load('cp.txt'); % ph in deg.
wn = find(w>130); % wn = find(w>133);
w = w(1:wn(1));
mag = mag(1:wn(1));
ph = ph(1:wn(1));

figure(2);clf;
subplot(211); semilogx(w,mag,'k'); xlabel('Freq. (Hz)'); ylabel('Mag. (dB)'); hold on
subplot(212); semilogx(w,ph,'k'); xlabel('Freq. (Hz)'); ylabel('Ph. (deg)');hold on

%complex freq response G (Transfer Function)-----
i = sqrt(-1);
for k =1:length(w)
    a = 10^(mag(k)/20)*cos(ph(k)*pi/180);
    b = 10^(mag(k)/20)*sin(ph(k)*pi/180);
    G(k,1) = a+b*i; % the Transfer Function of system
end

% make G even and get G_jq (conjugate of G)------
w = [0;w];
G = [G(1); G]; % Even G
G_jq = conj(G); % The conjugate of the even TF (G)
ome_g = w*2*pi; % get the omega of G_ff and G

% Create your optimal inversion code below -- please follow the process flow described in↙
homework.
%Step 1
%Compute Optimal Inversion of Transfer Function
Q(0<=w & w<130) = 1;
```

```
Q(w>130) = 0;
R(0<w & w<130) = 0;
R(w>130) = 1;
Q = Q';
R = R';

%Calculate Optimal Inversion Transfer Function
for i = 1:length(w)
    Gff(i,1) = (G_jq(i)*Q(i))/(R(i) + G_jq(i)*Q(i)*G(i));
end

% Step 2
% Cleaning up Yd for fft

[Mag, Phase, freq, Yd_FFT] = fft_sample(yd, 500);
figure
plot(freq,Mag)

G_interp = interp1(ome_g, G, freq*2*pi, "linear",0);
Gff_interp = interp1(ome_g, Gff, freq*2*pi, "linear",0);

G_Full = [G_interp,flip(conj(G_interp(1:end-1)))];
Gff_Full = [Gff_interp,flip(conj(Gff_interp(1:end-1)))];

Uopt_jw = Gff_Full .* Yd_FFT;
Yopt_jw = G_Full .* Uopt_jw;

uopt = real(ifft(Uopt_jw));
yd_opt = real(ifft(Yopt_jw));

numSec = 3/SignalFreq;

cycIdx1 = find(t==(tend-numSec));
cycIdx2 = find(t==(tend));

figure
subplot(2,1,1)
hold on
plot(t(cycIdx1:cycIdx2), yd(cycIdx1:cycIdx2), "k")
plot(t(cycIdx1:cycIdx2), yd_opt(cycIdx1:cycIdx2), "--r")
legend("$y_d(t)$", "$y_{opt}(t)$", "Interpreter", "Latex")
xlabel("Time"); ylabel("Respone"); title("Desired and Optimal Trajectory vs Time [10 Hz
Signal]")
subplot(2,1,2)
plot(t(cycIdx1:cycIdx2),uopt(cycIdx1:cycIdx2))
xlabel("Time"); ylabel("Input"); title("Optimal Input vs Time [10 Hz Signal]")

%% 18Hz
clear, clc, close all
% load desired input trajectory signal---------
load yd01.in % trajectory for 1 Hz
```

```
yd = yd01(1:29999);
SignalFreq = 18; % Hz
SamplingFreq = 500; % Hz
dt = 1/SamplingFreq;
t = dt*[0:length(yd)-1];% time(s)--signal length
yd = interp1(t, yd, t.*SignalFreq, 'linear', 0);
figure(1);clf;
subplot(211); plot(t,yd); xlabel('time (s)'); ylabel('y_d')

tstart = 15/SignalFreq;
tend = 45/SignalFreq;

% [~,I1] = min(abs(t-0.833));
% I2 = find(t==tend);
I  = find(yd);
I1 = I(1) - 1;
I2 = I(end) + 1;
t = t(I1:I2);
yd = yd(I1:I2);


%=======================================================
% system modeling for (Transfer Function) G
%system modeling for (Transfer Function) G
%=======================================================
w = load('cm.x'); % load frequency
mag = load('cm.txt'); % load mag in dB
ph = load('cp.txt'); % ph in deg.
wn = find(w>130); % wn = find(w>133);
w = w(1:wn(1));
mag = mag(1:wn(1));
ph = ph(1:wn(1));

figure(2);clf;
subplot(211); semilogx(w,mag,'k'); xlabel('Freq. (Hz)'); ylabel('Mag. (dB)'); hold on
subplot(212); semilogx(w,ph,'k'); xlabel('Freq. (Hz)'); ylabel('Ph. (deg)');hold on

%complex freq response G (Transfer Function)-----
i = sqrt(-1);
for k =1:length(w)
    a = 10^(mag(k)/20)*cos(ph(k)*pi/180);
    b = 10^(mag(k)/20)*sin(ph(k)*pi/180);
    G(k,1) = a+b*i; % the Transfer Function of system
end

% make G even and get G_jq (conjugate of G)------
w = [0;w];
G = [G(1); G]; % Even G
G_jq = conj(G); % The conjugate of the even TF (G)
ome_g = w*2*pi; % get the omega of G_ff and G
```

```
% Create your optimal inversion code below -- please follow the process flow described in
homework.
%Step 1
%Compute Optimal Inversion of Transfer Function
Q(0<=w & w<130) = 1;
Q(w>130) = 0;
R(0<w & w<130) = 0;
R(w>130) = 1;
Q = Q';
R = R';

%Calculate Optimal Inversion Transfer Function
for i = 1:length(w)
    Gff(i,1) = (G_jq(i)*Q(i))/(R(i) + G_jq(i)*Q(i)*G(i));
end

% Step 2
% Cleaning up Yd for fft

[Mag, Phase, freq, Yd_FFT] = fft_sample(yd, 500);
figure
plot(freq,Mag)

G_interp = interp1(ome_g, G, freq*2*pi, "linear",0);
Gff_interp = interp1(ome_g, Gff, freq*2*pi, "linear",0);

G_Full = [G_interp,flip(conj(G_interp(1:end-1)))];
Gff_Full = [Gff_interp,flip(conj(Gff_interp(1:end-1)))];

Uopt_jw = Gff_Full .* Yd_FFT;
Yopt_jw = G_Full .* Uopt_jw;

uopt = real(ifft(Uopt_jw));
yd_opt = real(ifft(Yopt_jw));

numSec = 3/SignalFreq;

[~,cycIdx1] = min(abs(t-(tend-numSec)));
[~,cycIdx2] = min(abs(t-(tend)));

figure
subplot(2,1,1)
hold on
plot(t(cycIdx1:cycIdx2), yd(cycIdx1:cycIdx2), "k")
plot(t(cycIdx1:cycIdx2), yd_opt(cycIdx1:cycIdx2), "--r")
legend("$y_d(t)$", "$y_{opt}(t)$", "Interpreter", "Latex")
xlabel("Time"); ylabel("Respone"); title("Desired and Optimal Trajectory vs Time [18 Hz
Signal]")
subplot(2,1,2)
plot(t(cycIdx1:cycIdx2),uopt(cycIdx1:cycIdx2))
xlabel("Time"); ylabel("Input"); title("Optimal Input vs Time [18 Hz Signal]")
```

```
function [ Mag, Phase, freq, G] = fft_sample( data, sample_rate )
%Function does a fast fourier transform by inputing data(single sample set
%of  a signal) and sample rate (Sampling frequency in Hz) and outputting
%Magnitude(Volts), Phase(degrees), and frequency(Hz).

%Compute DFT using fft function
G = fft(data);

%Smallest measurable frequency
spectralResolution = sample_rate/length(data);

%Largest measurable frequency
f_Nyquist = sample_rate/2;

%Define range of frequencies
freq = 0:spectralResolution:f_Nyquist;

%Calculate magnitude in volts
Mag = abs(G(1:length(freq)))/length(data);
Mag(2:end-1) = 2*Mag(2:end-1);

%Calculate power of frequencies
Power = Mag.^2;

%Calculate Phase
Phase = angle(G)*180/pi;
end
```